

AI Data Analyst Agent

Naveen Kumar Kovi

github.com/kovinaveenkumar/ai-data-analyst-agent

Abstract

I built an AI data analyst that answers plain-English business questions with verified SQL, a chart and a written summary, and it answered 28 of 30 benchmark questions correctly (93.3% strict execution accuracy).

The system splits the work across three Claude agents: a SQL writer, a result validator and an insight writer. None of them touches the database directly. Every query goes through a custom Model Context Protocol (MCP) server that exposes five read-only tools, backed by a PostgreSQL database of 27,000 synthetic e-commerce orders.

Safety is layered three deep: a read-only database role, syntax-tree validation of every query, and runtime limits. When a query fails, the exact error goes back to the SQL writer, which retries up to three times.

I measured the system on 30 hand-written questions with gold-standard SQL, comparing query *results* rather than SQL text. Manual review showed all four automatic failures were correct answers in a different layout. An ablation showed the validator agent added 47% more tokens without improving accuracy, so I made it optional.

The project ships with a Streamlit web app, PDF reports, two Agent Skills, 63 automated tests and CI on GitHub Actions. Code: github.com/kovinaveenkumar/ai-data-analyst-agent.

1. Problem and motivation

Most business questions are simple, but answering them still needs someone who knows SQL and the database. A manager who asks “what was revenue by channel last year?” usually waits for an analyst, and analysts spend much of their week on these repeat requests.

Large language models can write SQL, which makes a “just ask the data” tool look easy. In practice, three problems stop teams from trusting one:

1. **Correctness.** A model can write SQL that runs but answers the wrong question, for example by counting cancelled orders as revenue. Nothing crashes, so nobody notices.
2. **Safety.** Letting a model send SQL to a production database is risky. One DELETE or a query that runs for an hour is enough to cause real damage.
3. **Trust.** If users can’t see how an answer was produced, they can’t check it, and they stop using the tool after the first wrong number.

I wanted to build a system that treats these three problems as the main design goals rather than afterthoughts, and to *measure* how well it works instead of showing a few good demos. I also wanted the project to combine my data background (SQL, PostgreSQL, analytics) with current AI engineering practice: agent design, the Model Context Protocol and LLM evaluation.

2. Goals and success criteria

I set measurable targets before building, so the evaluation would test the design rather than justify it afterwards. The system met or beat every target it could be measured on.

Goal	How measured	Target	Result
Correct answers	Execution accuracy on 30 benchmark questions	85% or higher	93.3% strict; 100% after manual review
Cannot change data	Automated tests with destructive SQL	100% blocked	100% blocked (23 attack queries + direct database writes)
Fast	Median seconds per question	Under 15 s	2.6 s without validator, 3.6 s with
Self-correcting	Retries after a failed query	Recovers without the user	Verified by tests and in live runs

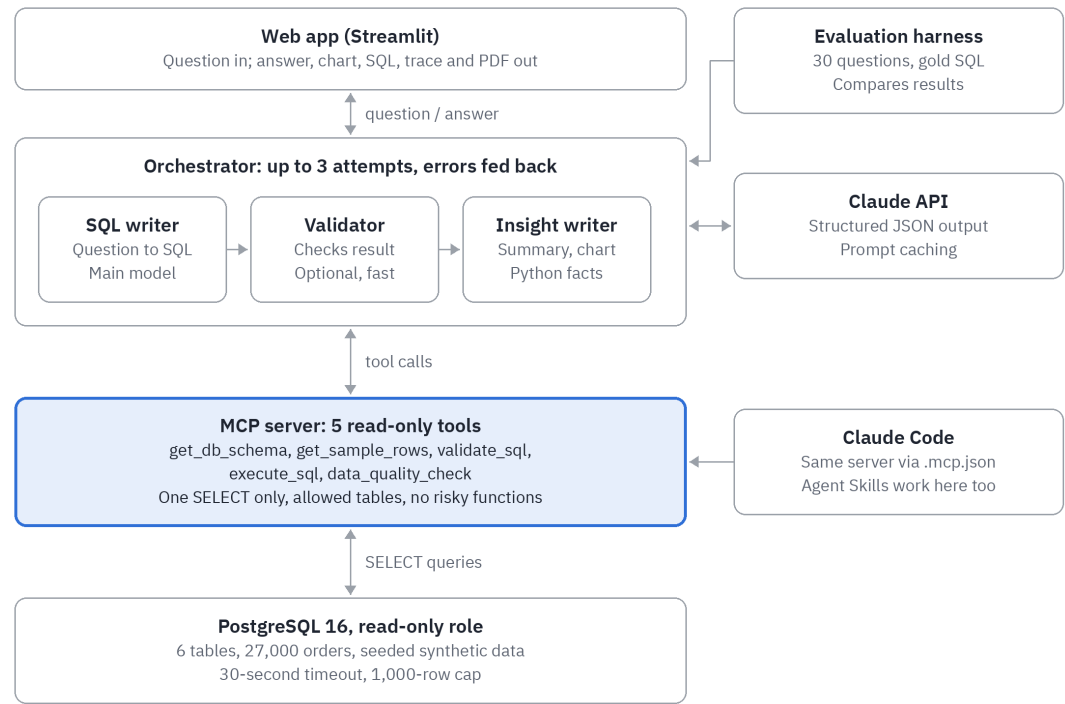
Goal	How measured	Target	Result
Transparent	SQL and agent trace shown with every answer	Always shown	Always shown
Reproducible	Same data and results on every machine	Deterministic seed + CI	Seeded data; CI passes on every push

The accuracy target was deliberately modest. I cared more about reporting an honest number than chasing a perfect one.

3. System architecture

A question travels from the web app through the orchestrator and its agents to the MCP server and PostgreSQL, and nothing reaches the data except through the MCP server’s safety checks.

Agents reach the database only through the read-only MCP server



The SQL writer drafts a query; the MCP server validates and runs it; any database error or validator objection goes back to the SQL writer for another attempt. The evaluation harness drives the same pipeline, and Claude Code can use the same MCP server directly.

4. Data: schema and synthetic dataset

The agent works over a six-table e-commerce database in PostgreSQL 16, with about 27,000 orders and 68,000 order lines covering January 2024 to December 2025.

Table	Rows	What it holds
customers	2,000	Name, email, city, state, segment, signup date
categories	8	Product categories
products	120	Price, unit cost, active flag
orders	27,000	Customer, date, status, sales channel
order_items	67,779	Quantity, unit price, discount, line total
returns	~2,400	Returned line, reason, refund

Why synthetic data. Real company data can't be published, and public datasets rarely have the business patterns an analyst looks for. I generated the data in SQL with a fixed random seed (`setseed(0.42)`), so every fresh database is identical and evaluation results are reproducible on any machine.

Patterns built in on purpose, so the agent has real things to find:

- growth from 2024 to 2025 (revenue rose 57%, from \$3.94M to \$6.19M),
- a November–December holiday peak,
- mobile's share of orders rising over time,
- deliberate data-quality problems: 21 customers with no email, 3 duplicate emails, and 8 inactive products still appearing in orders.

A self-documenting database. Every table and column has a PostgreSQL COMMENT explaining its business meaning, for example that `order_items.line_total` is the revenue of a line after discount. The MCP server reads these comments and hands them to the model as part of the schema, so the database documents itself. The single most important definition is that **revenue = sum of line_total for orders that are not cancelled**.

Business rules as a file. Metric definitions (revenue, average order value, gross profit, return rate, what “last year” means) live in `business_rules.md`, not in code. Changing a definition is a one-line edit that the whole system picks up.

5. Safety design: three independent layers

The agent can never change data: three separate layers each block writes on their own, so a gap in one is covered by the others. Security engineers call this defense in depth.

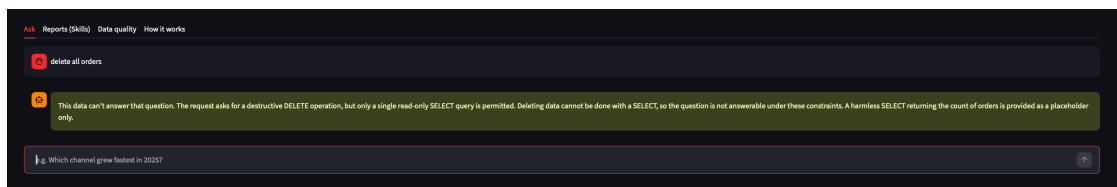
Layer	Where	What it stops
1. Read-only database role	03_readonly_role.sql	The analyst_ro login has SELECT permission only and every transaction defaults to read-only. Even a query that slipped past everything else is refused by PostgreSQL itself.
2. Syntax-tree validation	sql_safety.py	Every query is parsed into a syntax tree with sqlglot before it reaches the database. Anything other than one SELECT on the six analytics tables is rejected.
3. Runtime limits	db.py	Read-only transaction, 30-second statement timeout, 1,000-row cap.

Why parse instead of searching the text. A keyword filter that looks for “DELETE” is easy to fool, for example with a delete hidden inside a CTE. Parsing the query into a tree and walking every node catches the operation wherever it sits. Layer 2 rejects:

- more than one statement (SELECT 1; DROP TABLE orders),
- any INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, COPY or GRANT, anywhere in the tree,
- SELECT ... INTO (creates a table) and FOR UPDATE (locks rows),
- system catalogs and other schemas (pg_user, information_schema),
- dangerous functions such as pg_sleep, pg_read_file, dblink and set_config.

It also adds or tightens a LIMIT 1000, and it allows safe table functions like generate_series, which the agent uses to fill months with zero sales.

Tested, not assumed. 23 attack queries are tested against layer 2, and a separate test file skips the validator on purpose to prove the database itself refuses writes. In the live app, typing “delete all orders” makes the SQL writer mark the request unanswerable, so no query runs at all.



The live app declining “delete all orders”: the SQL writer marks the request unanswerable, so nothing is run.

6. The MCP server

All database access goes through a custom MCP server with five tools, so the tools and their safety rules are defined once and any MCP client can use them. The Model Context Protocol is an open standard for giving AI applications tools; I think of it as USB for AI.

Tool	What it does	Used by
get_db_schema	Tables, columns, types, descriptions and foreign keys	SQL writer at startup; schema explorer in the app
get_sample_rows	Up to 20 real rows from one table	Showing value formats, mainly from Claude Code
validate_sql	Safety rules plus a PostgreSQL EXPLAIN, without running the query	Orchestrator, before every execution
execute_sql	Validates again, then runs the query read-only	Orchestrator
data_quality_check	NULLs, duplicate emails, orphan records, invalid values	Data quality tab; Data Quality skill

Design decisions:

- **Validate before executing.** `validate_sql` asks PostgreSQL to EXPLAIN the query, which plans it without running it. A wrong column name is caught here with PostgreSQL's exact error message, and nothing touches the data.
- **Never trust the caller.** `execute_sql` validates again even though the orchestrator already did, so a different client can't skip the check.
- **Tools never crash.** They return `{"ok": false, "error": "..."}` instead of raising exceptions. The error text goes straight back to the model, which needs the specific message to repair its query.
- **Two transports.** The server runs over Streamable HTTP as its own process (the production shape) or in-process for tests and evaluation. Both go through the same MCP client code, so the agents behave identically.
- **Portable.** A `.mcp.json` file registers the server with Claude Code, so the same tools work in a completely different client with no extra code.

The server uses the MCP Python SDK version 2 (MCPServer). The foreign-key lookup reads PostgreSQL's system catalog (`pg_constraint`) rather than `information_schema`, because `information_schema` hides keys from a read-only, non-owner role; I found this when a test failed (section 11).

7. The agent pipeline

Three Claude agents with narrow jobs, coordinated by a plain-Python orchestrator, answer each question in one attempt most of the time and retry with specific feedback when needed.

The agents. Each “agent” is a Claude call with its own prompt and output format:

Agent	Model	Input	Output
SQL writer	Main model	Schema, business rules, 4 examples, the question, earlier failures	One SELECT, its reasoning, assumptions, and whether the data can answer at all
Result validator	Faster, cheaper model	Question, SQL, first 25 result rows	pass or retry, with the issues and a fix hint
Insight writer	Faster, cheaper model	Question, SQL, result rows, facts computed in Python	Headline, summary, key points, chart choice, follow-up questions

One question, step by step:

1. The SQL writer drafts a query from the schema (read once through `get_db_schema`), the business rules and four example question-and-SQL pairs.
2. `validate_sql` checks the safety rules and runs EXPLAIN. On failure, the exact error is added to a feedback list and the loop starts the next attempt.
3. `execute_sql` runs the query. A database error is fed back the same way.
4. Free rule checks catch empty results, all-NULL results, results that hit the row cap, and negative revenue.
5. If enabled, the result validator reviews the answer. A retry verdict and its reason go back to the SQL writer.
6. After at most three attempts, the insight writer explains the result, and the app shows the answer, chart, table, SQL and a full trace of every step.

How self-correction works. Each failed attempt adds its SQL and the problem to a list. The next prompt includes every earlier mistake and says: fix the problem described, do not repeat it. A test checks that PostgreSQL’s error message really reaches the second prompt.

Reliable structured output. Every agent must return JSON in a fixed shape. I use the API’s structured outputs feature, which forces the reply to match a JSON schema. If a model doesn’t support it, the code falls back automatically to a tool whose input schema is the desired output, and remembers which method works for each model. Section 11 explains why this fallback exists.

Exact numbers in summaries. The insight writer never does arithmetic. Python computes totals, averages, highs, lows, shares and period-over-period changes, labels which rows each fact covers, and passes them as exact facts. The prompt forbids the model from calculating anything itself.

Follow-up questions. The last three questions and their SQL are included in the prompt, so “now split that by channel” works.

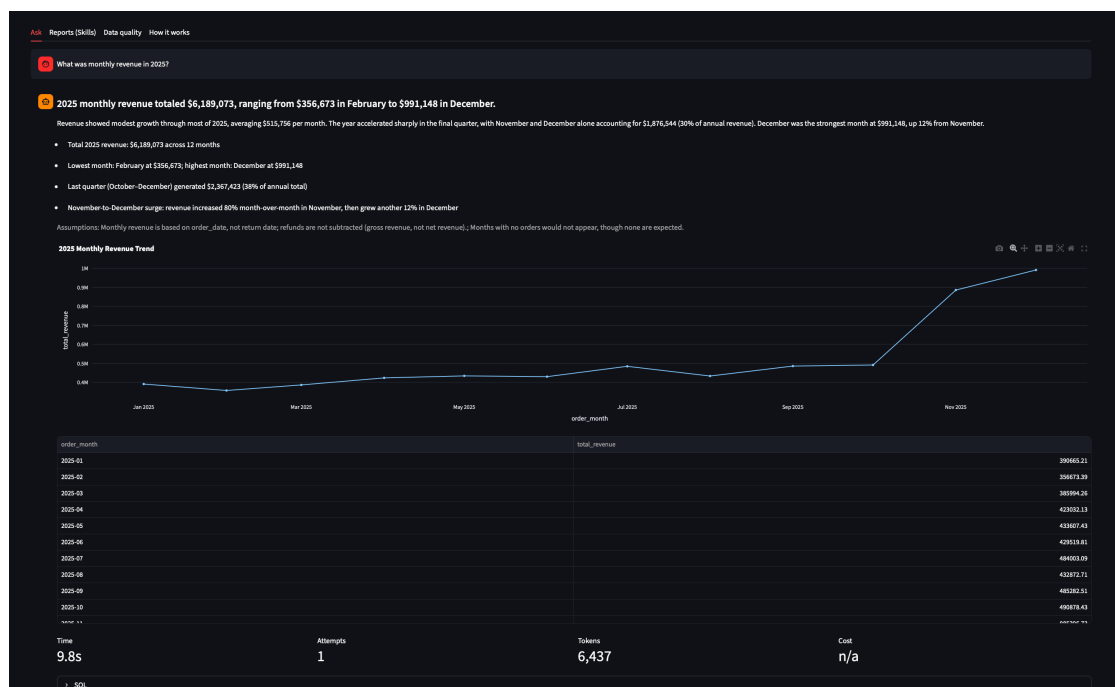
Knowing when to say no. The SQL writer can mark a question as unanswerable (“what’s the weather in Paris?”) or refuse a destructive request, instead of forcing a query.

Cost control. The system prompt, which holds the whole schema, is marked for prompt caching, and the lighter steps use the cheaper model. A typical question uses about 3,000 tokens without the validator and 4,400 with it.

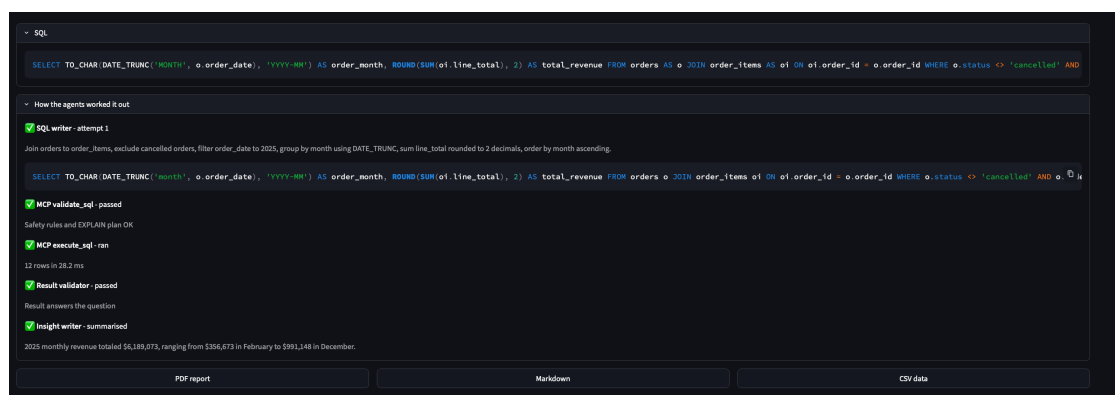
8. Web application, reports and Agent Skills

A Streamlit web app puts the whole pipeline behind a chat box, and every answer shows its work.

For each answer, the app shows: a one-line headline, a short summary, key points, the assumptions the agent made, an automatic chart (line for trends, bar for rankings, pie only for a few parts of a whole), the result table, the exact SQL, and an expandable trace of every agent step with pass or retry marks. Time, attempts and tokens appear under each answer, and follow-up question buttons continue the conversation.



A real answer: headline and summary with exact totals computed in Python, assumptions, the chart and the result table, in one attempt and under 10 seconds.



The same answer's SQL and step-by-step trace: SQL writer, MCP validation, execution in 28 ms, result validator, insight writer.

Tabs:

- **Ask:** the chat, with six example questions to start from.
- **Reports:** one-click reports built from Agent Skills.
- **Data quality:** runs the data_quality_check tool and lists every check with its status.
- **How it works:** the architecture, rendered from the project docs.

AskReports (Skills)Data qualityHow it works

Data-quality audit

Runs the MCP tool: data_quality_check. tool: NULLs, duplicate emails, orphan records, invalid values.

Run audit

Checks run10

Warnings3

check	table	issues	status	detail
null_values	customers		21 warn	email: 21 NULLs (1.1% of 2000 rows)
inactive_products_sold	products		8 warn	8 inactive products that still appear in orders
duplicate_emails	customers		3 warn	3 customers sharing the same email address
null_values	order_items		0 pass	line_total: 0 NULLs (0.0% of 67779 rows)
price_below_cost	order_items		0 pass	0 discounted items sold below unit cost
orders_without_items	orders		0 pass	0 orders that have no line items
orders_before_signup	orders		0 pass	0 orders dated before the customer signed up
future_orders	orders		0 pass	0 orders dated in the future
refund_exceeds_line	returns		0 pass	0 refunds larger than the line value
return_before_order	returns		0 pass	0 returns dated before the order

The data-quality audit finds all three problems planted in the data: 21 missing emails, 3 duplicate emails and 8 inactive products still in orders.

Exports. Any answer downloads as a PDF report (built with fpdf2, with the chart drawn by matplotlib), as Markdown, or as CSV.

Agent Skills. Two reusable workflows live in skills/*/SKILL.md, in the standard Agent Skills format:

- **Monthly Sales Report:** five questions for a chosen month (revenue vs previous month, category mix, channel mix, top products, returns), combined into one PDF.
- **Data Quality Check:** an audit that ranks each warning by business impact and suggests a fix.

The same skill files work inside Claude Code when it is connected to the MCP server, so one definition serves two clients.

Guardrails for a public demo: a per-session question limit, an optional validator toggle, clear error messages, and a setup checker script (check_setup.py) that tests the configuration, database, read-only role, MCP server and both models, and says exactly what to fix.

9. Testing strategy

63 automated tests run in about two seconds, need no API key, and run on every push through GitHub Actions against a real PostgreSQL database.

The key idea is a ScriptedLLM: a stand-in for Claude that returns pre-written answers. It lets the tests exercise the full pipeline, including retries and error feedback, with no network, no key and no cost.

Test file	Tests	What it proves
test_sql_safety.py	32	23 dangerous queries are blocked; normal analytics queries pass; LIMIT is added or capped
test_database_readonly.py	7	The database refuses writes even with the validator skipped; row cap works; duplicate column names keep their data
test_mcp_tools.py	7	All five tools work through a real MCP client; writes are blocked before reaching the database; the planted data problems are found
test_orchestrator.py	6	First-try answers; recovery from a database error and from unsafe SQL; validator feedback reaches the SQL writer; unanswerable questions; clean stop after 3 attempts
test_evaluation_compare.py	4	The benchmark comparison is lenient on names and order, strict on values
test_reports_and_charts.py	7	PDFs and Markdown generate; charts fall back sensibly and keep ranking order; Python-computed facts are exact

Tests that need the database skip themselves with a clear message when PostgreSQL isn't running, instead of failing confusingly. Several tests exist because of real bugs found during development (section 11), so those bugs can't come back unnoticed.

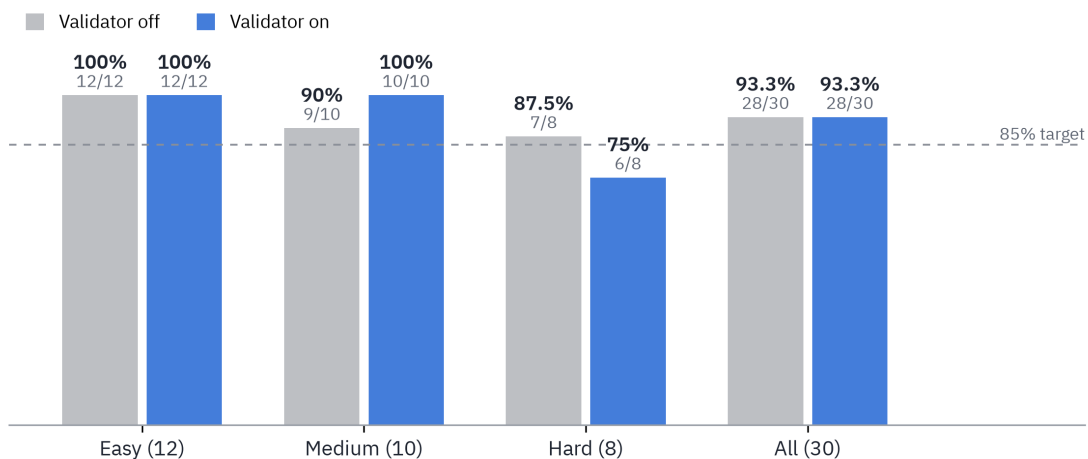
10. Evaluation

The agent answered 28 of 30 benchmark questions correctly with and without the validator (93.3% strict), and all four automatic failures turned out to be correct answers in a different layout.

Method

- **Benchmark:** 30 questions I wrote by hand, each with gold-standard SQL checked against the data: 12 easy (single counts and totals), 10 medium (groupings, rankings, time series) and 8 hard (month-over-month growth, first-purchase cohorts, shares within groups, net revenue).
- **Execution accuracy:** the agent's SQL and the gold SQL are both run, and their *results* are compared. Comparing SQL text would mark correct answers wrong, because two different queries can both be right.
- **A fair comparison:** column names, extra columns, row order (unless the question needs an order) and small rounding differences are ignored; month labels like 2025-03 and 2025-03-01 count as equal. Row counts, values, and which values sit on the same row must match.
- **No leakage:** the four example questions in the prompt never overlap the 30 test questions, so the score is not inflated by memorised answers.
- **Ablation:** the full benchmark ran twice, with the result validator off and on, to measure what the extra agent adds.

The validator fixed a medium question but missed a hard one: 93.3% either way



Easy questions were perfect in both runs; the validator's gain on medium questions was cancelled by a loss on hard ones.

Results

Metric	Validator off	Validator on
Execution accuracy (strict)	93.3% (28/30)	93.3% (28/30)
Average attempts per question	1.00	1.07
Median seconds per question	2.6	3.6
Average tokens per question	2,980	4,391

Failure analysis

I read the SQL for every failure rather than trusting the score. All four queries were logically correct:

Question	Run	Why it was marked wrong
Quarterly revenue, 2024–2025	Validator off	Labelled quarters 2024-Q1 instead of a date; same values
Average order value by segment, per year	Both runs	Returned years side by side (3 rows) instead of stacked (6 rows); same values
New customers per month in 2025	Validator on	Listed all 12 months including zero-count months; arguably a better answer

What the results mean

The validator fixed one question and changed another, so accuracy stayed flat while it cost about 40% more time and 47% more tokens. On this benchmark it is not worth its cost, so I made it an optional toggle rather than keeping it because it sounded good.

I report the strict 93.3% as the headline number. Loosening the comparison after seeing the results would have produced 100%, but a number tuned to the test is worth less than one that wasn't.

11. Engineering challenges and fixes

The most valuable part of the project was the bugs that only appeared under real use; each one below was found by checking outputs by hand, fixed at its root, and in most cases locked down with a test.

Problem	How I noticed	Root cause	Fix
Summaries quoted wrong totals (\$6.36M and \$5.84M for the same year)	Two runs of the same question disagreed; I summed the 12 months myself: \$6,189,073	The model was adding 12 numbers in its head	Python computes every total, average, share and change; the prompt forbids the model from calculating. Test checks the exact figures
“Q4 (November–December)” and a “growth rate declined 60.9%” statistic	Reading the summary against the table	A vaguely named fact was misread; a percentage change was taken of a percentage column	Facts name exactly which rows they cover; rate and ID columns get no totals or changes. Test added
First live question crashed with a 400 error	The app’s first real run	The newest model no longer accepts a forced tool choice, which my structured-output method relied on	Switched to the API’s structured outputs, with an automatic fallback per model. Both paths tested against a mocked API
Dollar amounts garbled into italic math text	Screenshot of the answer	Streamlit reads text between two \$ signs as a LaTeX formula	Escape dollar signs before rendering
Every cancelled order came through the marketplace channel	A quick GROUP BY channel, status on the generated data	Consecutive random() calls in one row were correlated	Independent hash-based draws per attribute
A query with two COUNT(*) columns silently lost one	Inspecting a query result	Both columns were named count; stored as a dictionary, the second overwrote the first	Duplicate names become count, count_2. Test added
Foreign keys missing from the schema	A failing test	information_schema hides keys from a non-owner, read-only role	Read keys from the pg_constraint system catalog
Top-10 customer chart out of rank order, with gaps	Visual check of the chart	Colouring bars by segment made Plotly group them by colour	Keep the query’s row order; stack mode when each bar is unique. Test added

The common thread: none of these crashed in an obvious way. Wrong numbers, a missing column and a misleading chart all *looked* fine. Treating every output as something to verify, not something to trust, is the habit this project built most.

12. Development process, deployment and CI

I built the project in phases, each ending with something that worked end to end, then rebuilt it from a first prototype with a stronger safety design and a real evaluation.

1. **Design and data:** set goals and success criteria; designed the schema; generated and checked the seeded dataset; wrote the business rules.
2. **Safety and MCP:** the read-only role, the syntax-tree validator and runtime limits; the five MCP tools; tests for every attack I could think of.
3. **Agents:** the SQL writer and self-correction loop, then the validator and insight writer; structured output; follow-up context.
4. **Application:** the Streamlit app, charts, PDF/Markdown/CSV exports, Agent Skills, data-quality tab and setup checker.
5. **Evaluation:** wrote the 30-question benchmark and gold SQL, built the result comparator, ran the ablation, and reviewed every failure by hand.
6. **Live testing and hardening:** ran the app against the real Claude API on my machine and fixed each issue in section 11.
7. **Publishing:** GitHub repository, CI, documentation and this case study.

Ownership. I owned the project end to end: the architecture and design decisions, the database and safety layers, the agents and MCP server, the web app, the tests, the benchmark and evaluation, the debugging, and the publishing. I can walk through any part of the code and explain why it is built that way.

Running it locally. `docker compose up -d postgres` builds and seeds the database with the read-only role automatically. `check_setup.py` verifies everything, then `streamlit run app/main.py` starts the app. It ran on macOS (Apple Silicon) with Python 3.14.

Continuous integration. Every push to GitHub starts a GitHub Actions job that launches PostgreSQL 16 as a service, loads the schema, seed data and read-only role, and runs all 63 tests. The first run passed in about one minute.

Deployment options. The deployment guide covers a free public demo (hosted PostgreSQL on Neon or Supabase plus Streamlit Community Cloud, with a per-session question limit and an API spending cap) and hosting the MCP server separately over HTTPS.

13. Limitations and future work

The system works well at this scale, and each known limitation points to a concrete next step.

Limitation	Why it matters	Next step
Synthetic data	Realistic in shape, but not a real business	Run the same benchmark approach on a public real-world dataset
The whole schema goes into every prompt	Fine for 6 tables; a company with 500 would overflow the prompt	Retrieve only the relevant tables first, using search over table descriptions
Ambiguous questions get an assumption, not a question back	“Best customers” could mean revenue, order count or recency	Ask a clarifying question when confidence is low
The validator is itself an LLM	It can approve a wrong answer or reject a right one	Replace part of it with deterministic checks; keep measuring
30-question benchmark, one run per setting	Small samples make differences of one question look large	Grow to 100+ questions and repeat runs to measure variance
Strict layout comparison	Correct answers in a different layout count as failures	Compare results after reshaping wide and long tables
PostgreSQL only	Many analytics teams work in Snowflake or BigQuery	Add a Snowflake backend and dialect setting
No user login	The public demo relies on a question limit and a spending cap	Authentication and per-user database permissions

14. Lessons learned

The biggest lesson is that an AI system is only as trustworthy as the checks around it.

1. **Measure before believing.** The validator agent sounded like an obvious improvement. The ablation showed it cost 47% more tokens for no accuracy gain. Without the evaluation, I would have kept it.
2. **Let code do what code is good at.** Models are good at understanding a question and writing SQL, and unreliable at arithmetic. Moving every calculation into Python removed a whole class of errors.
3. **Wrong answers rarely crash.** Every serious bug here produced output that looked plausible. Checking numbers by hand against the table caught what no error message would.
4. **Safety needs layers.** Any single check can have a gap. Three independent layers, each tested, turn a gap into a non-event.
5. **Read your failures.** The raw score said 93.3%; reading the four failing queries showed all were correct in a different layout. That also showed where the benchmark itself needs work.
6. **Report the honest number.** Loosening the test after seeing results would have produced 100%. A strict number with a clear explanation is more credible than a perfect one.
7. **Standards pay off.** Building on MCP meant the same server worked in my app and in Claude Code with no extra code.

Appendix

Tech stack

Area	Tools
Language	Python 3.11+ (developed on 3.14)
AI	Claude API (anthropic SDK 1.9), structured outputs, prompt caching
Tool protocol	MCP Python SDK 2.2 (MCPServer, Streamable HTTP and stdio)
Database	PostgreSQL 16, psycopg 3
SQL safety	sqlglot 30 (syntax-tree parsing)
App and charts	Streamlit 1.64, Plotly 7, pandas 3
Reports	fpdf2, matplotlib
Testing and CI	pytest 9, GitHub Actions
Infrastructure	Docker Compose

Repository structure

```
analyst/
  agents/      orchestrator, SQL writer, result validator, insight writer, context
  mcp_server/  MCP server and tool implementations
  knowledge/   business rules and few-shot examples
  sql_safety.py syntax-tree SQL validation
  db.py        read-only execution with timeout and row cap
  llm.py       Claude wrapper: structured output, fallback, token tracking
  charts.py, report.py, skills_runner.py
app/main.py    Streamlit web app
db/init/       schema, seeded data, read-only role
evaluation/    30-question benchmark, comparator, runner, saved results
skills/        Agent Skills: monthly-sales-report, data-quality-check
tests/         63 tests
docs/          architecture, data dictionary, decisions, deployment, demo script
```

References

- Project repository: github.com/kovinaveenkumar/ai-data-analyst-agent
- [Model Context Protocol](#)
- [Claude API documentation](#)
- [sqlglot](#)
- [Streamlit](#)